ALU LEGAL COMPANION

Technical Report

Elise Julio HAKIZIMANA | BSc Software Engineering | African Leadership University, Rwanda | 30 October 2026

Project Dimension	Summary and Verification Links
Live Deployment	https://alu-legal-companion-qu8w.onrender.com
API and Swagger Docs	https://alu-legal-companion.onrender.com/api/docs
Source Repository	https://github.com/EliseJulio/ALU-Legal-Companion
Scope	**Functional:** 11 of 13 completed (1 partial, 1 removed). **Non-functional:** 7 of 7 completed
Automated Tests	**2,003 tests:** 810 API integration, 996 UI component and 197 browser end-to-end. Browsers: Chromium, Firefox and WebKit
Code Size	135 source files with 15,226 lines (server 5,085 and client 10,141) and 154 test files with 19,577 lines
Stack	React 19, Vite and Tailwind CSS; Node.js, Express 4 and PostgreSQL; Render for hosting and Brevo for email

1. Overview

Rwanda publishes its laws publicly, but raw legal text rarely helps a student or staff member work out which office must hear their problem first. Approaching the wrong institution often means being turned away or missing a filing deadline. ALU Legal Companion is my answer to that problem, built for ALU students and staff. For each kind of legal matter the platform names the institution to approach first, cites the law, outlines the steps and states the deadline. Users can also track a matter and receive email reminders ahead of deadlines. They can ask a legal expert a question anonymously with a one-time access code. Nothing reaches the public until an administrator has reviewed and published it. Evidence below refers to file paths in the repository.

2. Scope Alignment

Tables 1 and 2 cover every functional (FR) and non-functional (NFR) requirement of the approved proposal. Tests live in `server/tests` and `client/tests`.

Table 1. Functional requirements of the proposal and their final status.

Planned Feature	Intended Purpose	Status	Repository Evidence
FR-01 Directory	Browse legal bodies by type, location and free service	**Completed**	`routes/directory.js` (filters `category`, `type`, `free`, `q`); `Directory.jsx`; `directory-providers.test.js`
FR-02 Routing	Name the institution to approach first, with law, steps and deadline	**Completed**	`services/routing.js`; `MatterRoute.jsx`; `matter-routes.test.js`
FR-03 Entry Details	Contact, languages, fee status, date last verified	**Completed**	Columns `languages`, `is_free`, `last_checked_at`; stale flag after 180 days
FR-04 Guide Library	Guides with topic filter, search, source law and disclaimer	**Completed**	`GuideLibrary.jsx`; `services/siteSearch.js`; `guides.test.js`; `search.test.js`
FR-05 Matters	Record steps with dates and see the next action	**Completed**	`routes/matters.js`; table `matter_events`; `matters.test.js`
FR-06 Reminders	Email before a deadline and record delivery	**Completed**	`services/reminders.js`; statuses sent, missed, cancelled; `reminders.test.js`; real email received from the live site
FR-07 Sharing	Share a matter with a legal expert for notes; revoke at any time	**Completed**	`POST` and `DELETE /matters/:id/share`; `matters-sharing.test.js`
FR-08 Anonymous Help	Questions and bookings with no identity, using a one-time code	**Completed**	`services/anonToken.js`; `questions.test.js`; `bookings.test.js`
FR-09 Accounts	Registration with email check, sign-in, reset, settings, saved guides	**Completed**	`routes/auth.js`; change of email added; `auth.test.js`; `change-email.test.js`
FR-10 Review and Publish	Content is checked and published; any edit returns it to draft	**Completed**	The administrator reviews and publishes. An edit resets the item to draft and clears the publisher. The database refuses a published row with no publisher. `guides.test.js`
FR-11 Expert and Admin Tools	Answer questions without learning who asked; manage users, invitations, directory, contacts	**Completed**	`routes/questions.js`; `routes/admin.js`; `admin-*.test.js`; emergency numbers on every page

Planned Feature	Intended Purpose	Status	Repository Evidence
FR-12 Assistant	Answer from published content with sources	**Removed**	Removed entirely. It was a very large piece of work and I had no prior knowledge of AI. No model is called anywhere in the code. A legal expert answers every question
FR-13 Evaluation Run	Score routing and assistant answers on test sets	**Partial**	The scoring script and test-set files exist in my local working copy and are not committed to this repository

Table 2. Non-functional requirements of the proposal and their final status.

Planned Feature	Intended Purpose	Status	Repository Evidence
NFR-01 Security	bcrypt cost 12, startup secret check, role checks, headers, CORS, rate limits	**Completed**	`passwords.js`; `index.js`; `helmet`; `CORS_ORIGINS`; `middleware/rateLimit.js`
NFR-02 Privacy	Anonymous stays anonymous; shared matters visible only while shared	**Completed**	Only a SHA-256 hash of the code is stored; `anon-token.test.js`; `matters-sharing.test.js`
NFR-03 Reliability	Automated tests on every commit	**Completed**	`.github/workflows/ci.yml` runs three jobs on every push
NFR-04 Performance	A guide page within 2 seconds on mobile	**Completed**	About 0.7 s on the live site when the server is awake. After idle the free plan took 22.8 s once (Section 4.4). Further tuning is planned
NFR-05 Usability	Phone layout, keyboard use, WCAG AA contrast	**Completed**	No horizontal scroll from 320 to 1440 px; contrast verified in `tokens.test.js`. The usability survey is still pending and layout polish is planned
NFR-06 Maintainability	Content editable without a new release	**Completed**	Admin pages for guides, entries, routes, checklists and emergency contacts
NFR-07 Accuracy	Publisher and date shown; old entries flagged	**Completed**	The publishing administrator and date are stored and shown; entries unchecked for 180 days are flagged

2.1 Changes to the Approved Scope

1. **The assistant was removed entirely (FR-12, FR-13).** The proposal included an assistant that answers from published guides and a script to score it. It was a very large piece of work and I had no prior knowledge of AI, so I removed it. Every question now goes to a legal expert and no answer is generated. The scoring script and test sets stay in my local working copy.
2. **Publishing moved to the administrator.** In the proposal legal experts reviewed and published guides. Now the administrator writes, verifies, publishes and withdraws all guides, directory entries, routes and checklists. The legal expert has three jobs: answering questions (including anonymous ones), holding consultations through the Appointments page and reading shared student matters with their attached documents.
3. **Features added beyond the proposal.** Encrypted document upload with lawyer review and comments; checklists for procedures such as buying a car; a "how to do it" block in guides with fees, documents and offices; a map card that contacts no outside server until clicked; a verified change of email; a Swagger page for every route; live deployment on Render with email through Brevo.

3. Technical Implementation

3.1 Architecture and Data Model

The product has three tiers (Figure 1). A React 19 single-page application, built with Vite and styled with Tailwind CSS, calls a Node and Express 4 REST API. The API reads and writes PostgreSQL through plain parameterised SQL with no ORM. It has 13 route modules, 12 service modules and 86 operations. Twenty-six numbered migrations build 17 tables. The client has 41 pages and 34 shared components. There are four roles: student or staff, legal expert, administrator and visitor. Every protected route calls `requireAuth` with its allowed roles and the role is read from the database on every request, so a role change or sign-out takes effect at once.

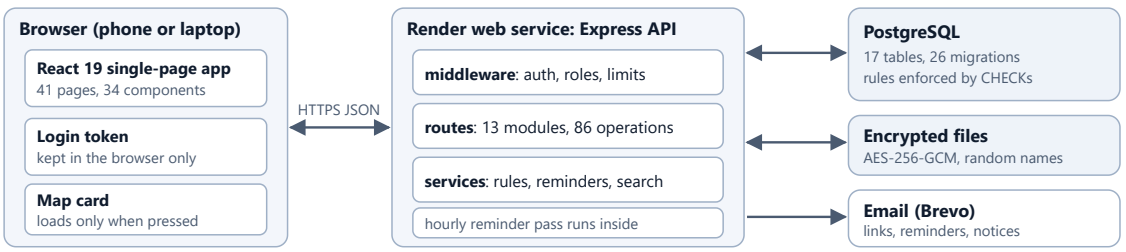


Figure 1. Three tiers. Only the API touches the database, the encrypted files and the mail service.

Table 3. What is stored and where.

Data	Stored where	Notes
Name, email, password	Table `users`	Only a bcrypt hash of the password. No phone number or home address is collected
Matters, steps, reminders	Tables `matters`, `matter_events`, `reminders`	Seen by the owner and by the expert a matter is shared with, only while it is shared
Anonymous questions and bookings	Tables `questions`, `bookings`	No name or email. Only a hash of the one-time code is kept
Attached documents	Encrypted files on the server, details in `matter_documents`	AES-256-GCM, random file names, deleted only by their owner
Guides, entries, routes, checklists, contacts	PostgreSQL	Edited by the administrator without a new release
Words typed in the search	Not stored	The search reads published content only

3.2 How the Core Mechanisms Work

Table 4. Core mechanisms.

Mechanism	How it works
Publishing rule	Editing a published guide, entry or route sets it back to draft and clears its publisher. A `CHECK` constraint makes a published row with no publisher impossible even if the code has a fault. The editor sends nothing until the administrator ticks "I understand".
Routing search	Typed text becomes a PostgreSQL `tsquery` with words joined by OR, because students describe a situation in a sentence and requiring every word would find almost nothing. `ts_rank` with length normalisation stops a long title winning by length. A route is listed only when the route and its first office are both published.
Reminders	Due dates are worked out on the server in Kigali time (UTC+2). Reminders go 7 and 1 days before. A send date already past moves to today and two on one day merge. An hourly pass sends what is due and is safe to repeat because a reminder leaves the pending state only once. A reminder is never sent after its deadline. It becomes missed. The email gives the date and never the matter title.
Anonymous access code	80 random bits written as four groups of four characters from an alphabet without look-alike letters. Typed input is normalised (O becomes 0). Only a SHA-256 hash is stored, so a leaked database cannot open anyone's record.
Document encryption	The file type is read from the leading bytes. The file is encrypted at rest with AES-256-GCM and its stored name is bound in as additional authenticated data, so a copied file cannot be read as another document. With no key configured the service refuses to store anything.

Verified change of email works in five steps. The same pattern protects sign-in sessions.

1. The settings form posts the new address and the current password to `POST /api/auth/change-email`.
2. The server checks the password and the ALU domain rule, keeps the new address as pending and emails a link to it. The old address gets a short notice with no link.
3. The reply is identical whether or not the address is already registered, so nobody can probe for accounts.
4. The link opens `/confirm-email-change`, which posts its token to `POST /api/auth/confirm-email-change`.
5. One database update swaps the address, marks it confirmed and raises the session counter. Every older session ends and the browser receives a new one.

4. Testing and Validation

4.1 Automated Testing Strategies

GitHub Actions runs all three tiers on every push (`.github/workflows/ci.yml`).

Table 5. Testing tiers, methods and what they found.

Strategy	Why It Was Chosen	How It Was Done	Results and Improvements
Integration tests: Jest and Supertest on real PostgreSQL	Core rules live in SQL constraints and Express middleware. A mock cannot break a constraint.	Each run builds a private schema from the migrations and drops it afterwards, so runs never collide. Requests go through the real Express app.	**810 tests in 45 suites pass.** Found: a wrong current password returned 401 and signed the user out. It now returns 400. I also broke the change-email code on purpose to check the tests: removing the session ending failed 1 test and writing the new address straight onto the account failed 15. The code was then restored.
Component tests: Vitest and Testing Library	Each page must show and send the right fields for each role.	The network is replaced by fixed replies. Any unplanned call fails the test.	**996 tests in 83 files pass.** Found: a stored link could carry a `javascript:` address, so pages now check a link before using it. Dates printed differently on a US-locale machine, so the format is now fixed to en-GB.
Browser tests: Playwright	Whole mobile journeys, touch targets, horizontal overflow.	A real browser at 390 px with the API stubbed so no data can change.	**197 tests in 17 files.** Found: two checks ran before the page had made its request, which failed on slower machines. They now wait for the request. Browser results in Section 4.3.

4.2 Boundary and Invalid Inputs

Table 6. Representative boundary and negative test cases.

Input Scenario	Expected	Observed Behaviour
Empty text, "not an address" or "two@@alustudent.com" as a new email	HTTP 400	Refused and nothing stored
Wrong current password when changing email	HTTP 400	Refused and the user stays signed in
A student asks for a gmail address; a legal expert asks for the same	400 and 200	The ALU domain rule applies by role
Email change to an address that already has an account	Same reply	Identical body and status; nothing sent
Verification link used twice, expired, of the wrong kind, made up or address taken meanwhile	HTTP 400	Each refused with a message saying what to do
Booking in the past; anonymous request without confirming the code was saved	Blocked	Refused before anything is sent (browser tests)

4.3 Environments, Browsers and Viewports

Table 7. Environments and cross-browser results.

Environment	Version	Observed Result
Developer machine	Windows 11, Node 24.15	810 server, 996 client and 197 browser tests pass
GitHub Actions	Ubuntu, Node 24, PostgreSQL 16 image	Runs on every push. Early runs exposed an old Node build, a date format difference and two timing races. All are fixed
Live deployment	Render free plan, PostgreSQL 18.6	Public pages and API respond (Table 8)
Chromium, 390 px	Playwright 1.64	**197 pass.** One request check raced under load in an earlier run. It now waits for the request
Firefox, 390 px	Playwright 1.64	**197 pass**
WebKit (Safari engine), 390 px	Playwright 1.64	**195 pass, 2 fail** on every run. Both check keyboard focus (Tab onto a link and onto a tick box). The cause is not yet investigated
Widths 320, 390, 834, 1440 px	Main pages	No horizontal scroll at any width

I tested desktop browser engines with a phone-sized window. I have not tested on physical phones, on Safari itself, with a screen reader or on older operating systems.

4.4 Live Performance Measurements

Table 8. Measurements on the live Render deployment (5 requests per target, from my network to the Frankfurt server).

Measurement Target	Observed Result
API: guides, routes, directory, emergency contacts	0.65 s to 0.90 s to first byte (about 0.70 s typical)
Website page (server awake)	0.39 s to 0.94 s
First request after idle (free plan wakes the server)	22.8 s once; every later request was fast
Main script bundle	491.8 kB (135.0 kB gzipped); admin pages load as separate chunks of 5 to 12 kB

The delay after idle comes from the free plan stopping idle servers. An always-on plan removes it. I ran no load test.

5. Code Quality

The server is split into `routes`, `services`, `middleware` and `migrations`. The client is split into `pages`, `components` and `lib`. Routes check who is asking and the shape of the input and services hold the rules. The largest hand-written file is 558 lines. A test compares the Swagger description with the real Express routes in both directions so the documentation cannot drift from the code. The client passes the oxlint linter and every schema change is a numbered migration committed in small steps. The code is ES modules of functions and React hooks and not class hierarchies, so the object-oriented principles appear through module boundaries.

Table 9. Object-oriented principles in modular JavaScript.

Principle	Where it shows
Encapsulation	`documentStore.js` exports store, read and delete. The key and the file layout stay inside. Only `passwords.js` hashes and checks passwords.
Abstraction	`createLimiter` builds every rate limiter from one definition. `recordReview` logs publishing decisions for guides, entries and routes.
Polymorphism	`siteSearch` picks a finder from a table of three libraries and calls each the same way.
Composition and reuse	34 shared components such as `PasswordField`, `DataTable` and `ErrorNotice` serve 41 pages. Inheritance is not used. Parts are combined instead.

Table 10. Algorithms and custom solutions.

Problem	Solution	File
Match a sentence to a route	OR-joined full-text query with length-normalised ranking	`services/routing.js`
Send reminders on time, once	Clamp and merge dates in Kigali time. An hourly pass with a one-way pending state	`services/reminders.js`
Anonymous return code	Base-32 encoding of 80 random bits, a forgiving normaliser and a stored SHA-256 hash	`services/anonToken.js`
Two uploads at once	The 25-document cap and file swap run under `pg_advisory_xact_lock`	`routes/matterDocuments.js`
Is a matter ready?	Each item is missing, pending, complete, needs fix or out of date. Ready only when every required file is complete	`services/checklists.js`
End old sessions	A session counter in the token and the user row. A change of password or email raises it	`routes/auth.js`

6. Limitations and Next Steps

The free Render plan sleeps when idle and its free database expires around 28 November 2026, so it must move to persistent hosting. The assistant is not part of the product. The usability survey, a load test and tests on physical phones and with screen readers are not done. The two WebKit keyboard-focus failures need a cause. Guide page speed and the phone layout can be tuned further. The legal content is sample content. Tests cannot show that a guide is correct, so a practising Rwandan lawyer should review it before real use.